

1 Softmax Gradient

1.1 Softmax K=2 Proof

Prove that with $K = 2$, $y_k = \frac{e^{z_k}}{\sum_{m=1}^K e^{z_m}}$

corresponds to $y = \frac{1}{1+e^{-z}}$. With $K = 2$, softmax becomes:

$$y_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} \cdot \frac{e^{z_1}}{e^{z_1} (1 + e^{z_2 - z_1})}$$

$$= \frac{1}{1 + e^{-(z_1 - z_2)}}$$

1.2 XOR Non-Linearly Separable

For data looking like $\begin{matrix} +A & -D \\ F & - \text{proof} \\ -B & +C \end{matrix}$

non-linearly separable:

- The decision boundary of a linear model separates inputs into two half-spaces.
- Each half-space is a convex set.
- For any 2 points in a convex set, the line segment connecting the two points is in the same convex set.
- AC is in the +ve half-space. So is F.
- BD is in the -ve half-space. So is F.
- F is in both, contradiction.

A good XOR feature map is

$$z = x_1 + x_2 - 2x_1x_2 - 0.5$$

2 Neural Networks

$$\mathbf{x} \rightarrow \mathbf{h}^{(1)} \rightarrow \mathbf{h}^{(2)} \rightarrow \mathbf{y}$$

$$\mathbf{h}^{(2)} = f^{(2)} \left(W^{(2)} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \right)$$

$$\mathbf{h}^{(L-1)} = f^{(L-1)} \left(W^{(L-1)} \mathbf{h}^{(L-2)} + \mathbf{b}^{(L-1)} \right)$$

$$\mathbf{z} = W^{(L)} \mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$$

$$\mathbf{y} = \text{softmax}(\mathbf{z})$$

2.1 Designing Two-Layer XOR

AND: $x_1 + x_2 - 1.5$, OR: $x_1 + x_2 - 0.5$

XOR: $(x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$ or $(x_1 \vee x_2) \wedge (\neg (x_1 \wedge x_2))$

2.2 Brute-Forcing

Binary inputs only: Hidden layer for each positive example should be a positive if input matches exactly, otherwise negative (makes sense to look like $[-N_1 + 0.5, -999, -999, 1, 1, -999]$). OR all the hidden layers together.

2.3 Forward Pass

For each $m = 1 \dots N$:

$$z_i^m = \sum_j W_{ij}^m a_j^{m-1} + b_i^m$$

$$a_i^m = \sigma^m(z_i^m)$$

For output layer N :

$$L = L(a^N, t)$$

2.4 Backward Pass

For output layer N :

$$\frac{\partial L}{\partial z_N} = \frac{\partial L}{\partial a^N} \cdot \sigma'^N(z^N)$$

For each $m = 1 \dots N$

$$\frac{\partial L}{\partial z_j^m} = \left(\sum_i \frac{\partial L}{\partial z_i^{m+1}} W_{ij}^{m+1} \right) \cdot \sigma'^m(z_j^m)$$

$$\frac{\partial L}{\partial W_{ij}^m} = \frac{\partial L}{\partial z_j^m} \cdot a_i^{m-1}$$

2.5 Vectorizing Calculations

Derivative of a matrix weight calls for an outer product, derivative for an activation function (vector $\rightarrow$ vector) calls for an element-wise product, derivative for a single-dimensional input calls for an inner product. **Double check the output dimension!**

$$M_{\uparrow H \times \overleftarrow{W}} = A_{\uparrow H \times \blacksquare} B_{\blacksquare \times \blacksquare} \dots Z_{\blacksquare \times \overleftarrow{W}}$$

Reminder: Matrix multiplication requires:

$$A_{H_{\text{opt}} \times Q} B_{\uparrow Q \times W_{\text{opt}}}$$

The height of the leftmost term in a chained matrix multiplication should match the height of the result. The width of the rightmost term should be the width of the result. Look out for: $\frac{\partial \mathcal{L}}{\partial a_j^{(1)}} = \sum_{k=1}^{D^{(2)}} \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} W_{kj}^{(2)} \Rightarrow$

$$\nabla_{\mathbf{a}^{(1)}} \mathcal{L} = (W^{(2)})^T (\nabla_{\mathbf{z}^{(2)}} \mathcal{L}) \text{ and } \frac{\partial \mathcal{L}}{\partial W_{ji}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(1)}} a_i^{(0)} \Rightarrow \nabla_{\mathbf{W}} \mathcal{L} = (\nabla_{\mathbf{z}^{(1)}} \mathcal{L}) (\mathbf{a}^{(0)})^T$$

2.6 Matrix Utils

$\exp(Z) 1_D = \text{sum squish} \rightarrow \leftarrow$
 $\text{np.sum}(\text{np.exp}(Z), \text{axis}=1)$
 $1_N \mathbf{a} = \text{np.sum}(\mathbf{a})$
 Also: $\mathbf{z}^{(1)} = W^{(1)} \mathbf{a}^{(0)} + \mathbf{b}^{(1)} \Rightarrow \mathbf{z}^{(1)}$
 $= A^{(0)} (W^{(1)})^T + 1_N (b^{(1)})^T$

$$\text{Also: } \begin{bmatrix} - & \mathbf{z}^T W & - \\ \vdots & \vdots & \vdots \end{bmatrix} = ZW$$

3 Bias-Variance Decomposition

Generalization Error = Bias + Variance + Bayes Error

- Bias: Model's inability to capture true relationship (underfitting)
- Variance: Model's sensitivity to random fluctuations (overfitting) – resample from the training set and the model changes by a lot

- Bayes error: Inherent noise in the problem Variables:

- $p_{\mathcal{D}}$ denotes the data-generating distribution. Sample a random training set $\mathcal{D}$ from it.
- A learning algorithm $\mathcal{A}$ takes $\mathcal{D}$ as input and produces a function $f: \mathbf{x} \rightarrow y$ as output.
- A test example $(\mathbf{x}, t)$ is sampled from $p_{\mathcal{D}}$ and a prediction $y = f(\mathbf{x})$ is made.
- A test error is computed through squared loss: $\mathcal{L}(y - t) = (y - t)^2 = f(\mathbf{x} - t)^2$
- Expected test error $\mathbb{E}[(y - t)^2] = \mathbb{E}[(f(\mathbf{x}) - t)^2]$ is analyzed

The average predictor $\bar{f}$ has that $\bar{f}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}} [f_{\mathcal{D}}(\mathbf{x})]$, the average of predictors on **all possible training sets** drawn from the same data generating distribution. The optimal predictor $\bar{y}$ is the expected target value at input $\mathbf{x}$, the best possible prediction for input $\mathbf{x}$ given the noisy data generating process, such that $\bar{y}(\mathbf{x}) = \mathbb{E}_t [t | \mathbf{x}]$

3.1 Decomposition

First: $\mathbb{E}_{\mathbf{x}, t, \mathcal{D}} [(f(\mathbf{x}) - t)^2]$

$$= \mathbb{E} \left[\left(f(\mathbf{x}) - \bar{f}(\mathbf{x}) + \bar{f}(\mathbf{x}) - t \right)^2 \right]$$

$$= \mathbb{E} \left[\left(f(\mathbf{x}) - \bar{f}(\mathbf{x}) \right)^2 \right] + 0 +$$

$$\mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - t \right)^2 \right]$$

Then: $\mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - t \right)^2 \right]$

$$= \mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x}) + \bar{y}(\mathbf{x}) - t \right)^2 \right]$$

$$= \mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x}) \right)^2 \right] +$$

$$\mathbb{E} \left[\left(\bar{y}(\mathbf{x}) - t \right)^2 \right]$$

3.2 Summarizing The Decomposition

$$\mathbb{E} [(f(\mathbf{x}) - t)^2] =$$

- $\mathbb{E}_{\mathbf{x}, \mathcal{D}} \left[\left(f(\mathbf{x}) - \bar{f}(\mathbf{x}) \right)^2 \right]$ (**variance**)
 - How much the predictions vary for different training sets
- $\mathbb{E}_{\mathbf{x}} \left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x}) \right)^2 \right]$ (**bias²**)
 - How wrong the avg. prediction is compared to expected target
- $\mathbb{E}_{\mathbf{x}, t} \left[\left(\bar{y}(\mathbf{x}) - t \right)^2 \right]$ (**bayes error**)
 - Noise inherent in the problem

3.3 Bagging

Bagging: Training on random subsamples and combine results. Done by creating multiple models from the training set re-sampled, computing the predictions for each, and averaging them: $y = \frac{1}{m} \sum_{i=1}^m y_i$. Bias is unchanged, since avg. prediction has the same expectation:

$$E[y] = E\left[\frac{1}{m} \sum_{i=1}^m y_i\right] = E[y_i]$$

Variance is decreased, since we are averaging over independent samples:

$$\begin{aligned} V[y] &= V\left[\frac{1}{m} \sum_{i=1}^m y_i\right] \\ &= \frac{1}{m^2} V\left[\sum_{i=1}^m y_i\right] \\ &= \frac{1}{m^2} \sum_{i=1}^m V[y_i] = \frac{1}{m} V[y_i] \end{aligned}$$

Bayes error is unchanged since we have no control over it.

Bootstrapping note: While bagging aims to reduce variance, it is not very helpful on highly correlated data, so maybe intentionally introduce randomness?

4 MLE

$$\begin{aligned} L(\text{data} | \theta) &= P(x_1 \dots x_N | \theta) \\ &= \prod_{i=1}^N P(x_i | \theta) \end{aligned}$$

Where $P(x_i | \theta)$ is the probability of x_i being that value assuming θ . For Bernoulli, it's $\theta^{x_i} (1 - \theta)^{1-x_i}$

To calculate MLE, take derivative with respect to θ and solve for 0.

$$\begin{aligned} \frac{A}{\theta} - \frac{B}{1-\theta} &= 0 \\ \Rightarrow \theta &= \frac{A}{A+B} \end{aligned}$$

4.1 Map Estimation

Max out $P(\theta | \text{data}) \propto P(\text{data} | \theta) \times P(\theta)$.

Beta dist: $P(\theta; a, b) \propto \theta^{a-1} (1 - \theta)^{b-1}$

Hence objective is to solve:

$$\arg \max_{\theta} P(\text{data} | \theta) P(\theta)$$

If θ holds more than one variable, if $P(\theta)$ is assumed over a distribution, then it's a product over all values that theta holds.

4.2 NB Classification

For an input: go over every possible output class and calculate the probability given that

class. The class that gives the highest is the one to pick.

$$\begin{aligned} p(c=1) &= \prod_{j=1}^D p(x_j | c=1) \\ p(c=0) &= \prod_{j=1}^D p(x_j | c=0) \end{aligned}$$

4.3 NB Learning

$$\begin{aligned} l(\text{data} | \theta) &= \sum_{i=1}^N \log p(c^{(i)} | \theta) \\ &+ \sum_{i=1}^N \sum_{j=1}^D \log p(x_j^{(i)} | c^{(i)}, \theta) \end{aligned}$$

A simple case would be

$$\begin{aligned} &= \sum_{i=1}^N \left(c^{(i)} \log(\pi) + (1 - c^{(i)}) \log(1 - \pi) \right) \\ &+ \sum_{i=1}^N c^{(i)} \sum_{j=1}^D \left(x_j^{(i)} \log \theta_{j1} + (1 - x_j^{(i)}) \log(1 - \theta_{j1}) \right) \\ &+ \sum_{i=1}^N (1 - c^{(i)}) \sum_{j=1}^D \left(x_j^{(i)} \log \theta_{j0} + (1 - x_j^{(i)}) \log(1 - \theta_{j0}) \right) \end{aligned}$$

And if using MAP for the θ values, we would add:

$$\begin{aligned} &+ (a - 1) \log(\theta_{j0}) \\ &+ (b - 1) \log(1 - \theta_{j0}) \\ &+ (a - 1) \log(\theta_{j1}) \\ &+ (b - 1) \log(1 - \theta_{j1}) \end{aligned}$$

Note: for multi-class, expect an expression that looks like this:

$$\sum_{n=1}^N \left(t_c^{(n)} \log(\pi_c) + \left(1 - \sum_{c=1}^{C-1} \pi_c \right) \right)$$

Note: When differentiating, beware of the gotcha – the probability of the j th feature and the i th data point given θ_{jc} is:

$$\begin{aligned} p(x_j^{(i)} | c^{(i)}) &= \left(p(x_j^{(i)} | c^{(i)} = 1) \right)^{c^{(i)}} \times \\ &\left(p(x_j^{(i)} | c^{(i)} = 0) \right)^{1-c^{(i)}} \end{aligned}$$

4.4 NB BOW Classification

$$\begin{aligned} \theta_{j1} &= \frac{\sum_{i=1}^N c^{(i)} x_j^{(i)}}{\sum_{i=1}^N c^{(i)}} \\ \theta_{j0} &= \frac{\sum_{i=1}^N (1 - c^{(i)}) x_j^{(i)}}{\sum_{i=1}^N (1 - c^{(i)})} \end{aligned}$$

5 GDA

$$\begin{aligned} N(\mathbf{x}; \mu, \Sigma) &= \frac{1}{(2\pi)^{\frac{D}{2}} |\Sigma|^{\frac{1}{2}}} \times \\ &\exp\left(-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu)\right) \end{aligned}$$

$$\begin{aligned} \hat{\mu}_k &= \frac{\sum_{i=1}^N r_k^{(i)} \mathbf{x}^{(i)}}{\sum_{i=1}^N r_k^{(i)}} \\ \hat{\Sigma}_k &= \frac{\sum_{i=1}^N r_k^{(i)} (\mathbf{x}^{(i)} - \hat{\mu}_k) (\mathbf{x}^{(i)} - \hat{\mu}_k)^T}{\sum_{i=1}^N r_k^{(i)}} \end{aligned}$$

5.1 Covariance Matrix

$A = Q\Lambda Q^T$ where Q holds the eigenvectors (column) and Λ holds the eigenvalues (correspond). COB to new basis requires $\tilde{\mathbf{x}} = Q^T \mathbf{x}$. Q is orthonormal, i.e. all vectors orthogonal and $|Q| = 1$ and $Q^{-1} = Q^T$

If Σ is not diagonal, the principal axes of the ellipses are aligned with the eigenvectors of Σ (eigenvectors give directions, eigenvalues give the stretch amount)

5.2 GDA Decision Boundary

For the binary case $c \in \{0, 1\}$, DB is $\log p(c_0 | \mathbf{x}) = \log p(c_1 | \mathbf{x})$. This requires

$p(c_k | \mathbf{x}) = \frac{p(\mathbf{x}|c_k)p(c_k)}{p(\mathbf{x})}$, where $p(\mathbf{x})$ can just be treated as a constant.

$$\begin{aligned} (\mathbf{x} - \mu_0)^T \Sigma_0^{-1} (\mathbf{x} - \mu_0) &= \\ (\mathbf{x} - \mu_1)^T \Sigma_1^{-1} (\mathbf{x} - \mu_1) + C & \end{aligned}$$

5.3 Shared Cov Implies Linear Decision Boundary

$$\begin{aligned} (\mathbf{x} - \mu_k)^T \Sigma_k^{-1} (\mathbf{x} - \mu_k) &= \\ (\mathbf{x} - \mu_j)^T \Sigma_j^{-1} (\mathbf{x} - \mu_j) + C & \end{aligned}$$

If $\Sigma_k^{-1} = \Sigma_j^{-1}$, then:

$$\begin{aligned} \mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mu_k^T \Sigma^{-1} \mathbf{x} &= \\ \mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mu_j^T \Sigma^{-1} \mathbf{x} & \end{aligned}$$

$$-2\mu_k^T \Sigma^{-1} \mathbf{x} = -2\mu_j^T \Sigma^{-1} \mathbf{x} + C$$

With shared variance, GDA is similar to logistic regression. Which one should we use?

- GDA makes a stronger modeling assumption.
- If the assumption is true, GDA is simpler and more efficient.
- If the assumption is not true (which happens very often), LR is better.